University of Chittagong
Department of Computer Science & Engineering
CSE 712: Compiler Lab

MiniCompiler

Design and Implementation of a MiniLang Compiler - Technical Report

A Flex/Bison compiler for MiniLang that performs lexical, syntactic, and semantic analysis, builds an AST, generates Three-Address Code (TAC), applies TAC-level optimizations, and emits simplified stack-machine target code.

1. Language Design Decisions

1.1 Overview of MiniLang Features Implemented

MiniLang, as implemented, supports exactly the constructs required by the assignment specification:

- **Variable declarations** for two scalar types, `int` and `bool`, with an optional initializer: `int x;` or `int x = 5;`
- **Assignment statements**: `x = expr;`
- **Arithmetic expressions** with `+ - * /`, standard precedence, and unary minus
- **Relational expressions** with `< > == !=`, producing a `bool` result
- `if`/`else` conditionals, including dangling-else resolution
- `while` loops
- `print(expr)` statements
- **Block scoping** with `{ }`, including shadowing of an outer variable by an inner declaration of the same name

1.2 Rationale for Simplifying Assumptions

- **Boolean literals.** The spec lists keywords `int, bool, if, else, while, print` but a usable `bool` type needs literal values, so `true` and `false` were added as keywords producing `BOOL_CONST` tokens. Without them, no MiniLang program could ever construct a boolean value directly.
- **No implicit type conversion.** Assignment and initialization require the expression type to exactly match the declared type (no int<->bool coercion). This keeps the type system simple and makes the required "type mismatch" semantic check meaningful and easy to trigger deterministically in test cases.
- **Approximate (not per-token) source locations.** Errors are reported using the lexer's current `yylineno` at the point a grammar rule reduces, rather than full `%locations` tracking with begin/end positions for every token. For a language with no multi-line expressions of real complexity, this gives accurate-enough line numbers while keeping the grammar actions simple.
- **Deterministic Makefile instead of the literal `*.c` glob.** The assignment's example build command, `gcc lex.yy.c parser.tab.c *.c -o minicompiler`, silently double-compiles `lex.yy.c` and `parser.tab.c` on a normal shell, because the `*.c` glob matches those two generated files a second time after they are already named explicitly on the command line - this produces duplicate-symbol link errors (duplicate `main`, `yylex`, etc.) on every modern GCC. The provided `Makefile` therefore lists every source file explicitly while still using exactly the three required tools (`flex`, `bison -d`, `gcc`) in the same roles.
- **Reserved `$`-prefixed internal names.** Compiler-generated temporaries (`$t1, $t2, ...`) and labels (`$L1, $L2, ...`) use a leading `$`, a character the lexer's identifier rule never accepts. This guarantees by construction that generated TAC names can never collide with a user-declared variable, without needing a runtime collision check.

2. Lexical Analysis

Implemented in `lexer.l` using Flex, with `%option yylineno` for automatic line tracking.

2.1 Regular Expressions and Token Definitions

```
DIGIT      [0-9]
LETTER     [a-zA-Z_]

"int"                    { return INT_KW; }
"bool"                   { return BOOL_KW; }
"if" | "else" | "while" | "print"    { return <the matching keyword token>; }
"true" | "false"         { yylval.ival = 1 or 0; return BOOL_CONST; }

{LETTER}({LETTER}|{DIGIT})*   { yylval.sval = strdup(yytext); return IDENT; }
{DIGIT}+                      { yylval.ival = atoi(yytext);  return INT_CONST; }

"==" "!=" "<" ">" "="          -> EQ, NE, LT, GT, ASSIGN
"+" "-" "*" "/"                -> PLUS, MINUS, STAR, SLASH
"(" ")" "{" "}" ";" ","        -> LPAREN, RPAREN, LBRACE, RBRACE, SEMI, COMMA
```

Keyword rules are listed *before* the general identifier rule. Flex resolves ties in matched length by rule order, so `"int"` and the identifier pattern both match the text `int` with equal length, but the keyword rule wins because it appears first - this is what lets keywords be recognized without a separate keyword-lookup table.

2.2 Handling of Whitespace, Comments, and Errors

- Whitespace (`[ \t\r]`) is discarded; newlines are matched separately so `yylineno` advances correctly.
- Line comments `// ...` are matched and discarded in one rule.
- Block comments `/* ... */` use a Flex start condition (`%x COMMENT`) so they can safely span multiple lines; an `<<EOF>>` rule reports an "unterminated comment" lexical error if the file ends before the comment is closed.
- Any other unrecognized character is caught by a catch-all `.` rule, which prints "Lexical Error at line N: unexpected character '...'" and increments a global `lexErrorCount`, then keeps scanning so multiple lexical errors in one file are all reported in a single pass instead of stopping at the first one.

2.3 Complexity Analysis: Time and Space Complexity of Scanning

Flex compiles all rules into a single deterministic finite automaton (DFA), so each input character is examined by a constant number of state transitions regardless of how many token rules exist. For an input of length *n* characters, scanning the entire source runs in **O(n)** time. Because MiniLang has no unbounded-lookahead constructs (no arbitrarily long lexemes requiring backtracking across the whole file), the DFA never needs to rescan already-consumed input. Space usage is **O(1)** with respect to input size for the automaton itself (its state table is fixed at generation time); the only per-token space is the `MAX_TOKEN_LEN`-bounded buffer for the currently matched lexeme, plus one `strdup`'d string per identifier token, which is O(1) additional space per token and is freed by the parser action that consumes it.

3. Syntax Analysis

Implemented in `parser.y` using Bison in LALR(1) mode, with `%define parse.error verbose` for descriptive syntax error messages.

3.1 Grammar Design and Precedence Rules

```
%left  EQ NE
%left  LT GT
%left  PLUS MINUS
%left  STAR SLASH
%right UMINUS
%nonassoc IFX
%nonassoc ELSE

program     : stmt_list
stmt_list   : /* empty */ | stmt stmt_list
stmt        : declaration | assignment | if_stmt | while_stmt | print_stmt | block
declaration : type IDENT SEMI | type IDENT ASSIGN expr SEMI
if_stmt     : IF LPAREN expr RPAREN stmt %prec IFX
            | IF LPAREN expr RPAREN stmt ELSE stmt
while_stmt  : WHILE LPAREN expr RPAREN stmt
block       : LBRACE stmt_list RBRACE
expr        : expr (PLUS|MINUS|STAR|SLASH|LT|GT|EQ|NE) expr
            | MINUS expr %prec UMINUS
            | LPAREN expr RPAREN | IDENT | INT_CONST | BOOL_CONST
```

Operator precedence and associativity are resolved declaratively with Bison's precedence table rather than by restructuring the grammar into a precedence-climbing cascade of nonterminals (e.g. `expr -> term -> factor`). This keeps the grammar itself small (a single `expr` nonterminal) while still deterministically resolving every arithmetic/relational ambiguity: multiplication/division bind tighter than addition/subtraction, which bind tighter than relational operators, and unary minus binds tightest of all (declared last among the arithmetic operators, giving it the highest precedence).

3.2 Conflict Resolution Strategy

Two categories of shift/reduce ambiguity are resolved explicitly:

- **Operator ambiguity** (e.g. is `a - b - c` `(a-b)-c` or `a-(b-c)`?) is resolved by the `%left`/`%right` precedence declarations above.
- **The dangling-else problem** (does `else` in `if (a) if (b) s1 else s2` bind to the inner or outer `if`?) is resolved with the classic Bison idiom: a pseudo-token `IFX` is given lower precedence than `ELSE`, and the `if`-without-`else` rule is tagged `%prec IFX`. Because `ELSE` has the higher precedence, Bison prefers to *shift* the `else` token rather than reduce the shorter rule, which binds `else` to the nearest unmatched `if` - the conventional and expected behavior. Building with `bison -d parser.y` reports zero unresolved conflicts.

3.3 Complexity Analysis: Cost of Parsing

Bison generates a table-driven LALR(1) shift-reduce parser. Each input token triggers exactly one shift and, amortized, at most a small constant number of reduce actions (bounded by the grammar's maximum rule length, which is fixed at compile-generation time and independent of input size). Consequently, parsing runs in **O(n)** time for a token stream of length *n*. The parser stack grows with nesting depth (e.g.

nested blocks, nested parenthesized expressions), so space usage is **O(d)** where *d* is the maximum nesting depth of the program - in the worst case O(n) for a pathologically deep single expression, but O(1) additional space per token in the common case of shallow nesting.

4. Abstract Syntax Tree (AST)

4.1 AST Structure and Node Types

`ast.h` defines a single tagged-union node type (`ASTNode`) discriminated by a `NodeType` enum, with a `C` union holding only the fields relevant to that node kind:

- **Leaves:** `NODE_INT_LIT, NODE_BOOL_LIT, NODE_IDENT`
- **Expressions:** `NODE_BINOP` (op, left, right), `NODE_UNARY_MINUS` (expr)
- **Statements:** `NODE_VAR_DECL` (type, name, optional init), `NODE_ASSIGN` (name, expr), `NODE_IF` (cond, then, optional else), `NODE_WHILE` (cond, body), `NODE_PRINT` (expr), `NODE_BLOCK` (stmt list)
- **Sequencing:** `NODE_STMT_LIST`, a singly-linked list node (stmt, next) used to chain statements within a block or at the top level

Every node also carries a source `line` number, propagated from `yylineno` at the point the corresponding grammar rule reduced, so that later phases (semantic analysis) can report errors against the original source line.

4.2 Why an AST Instead of a Parse Tree

A full concrete parse tree would retain a node for every grammar symbol matched, including punctuation tokens (`( ) { } ;`) and the intermediate nonterminals introduced purely to encode precedence (there are none of those here, since precedence is handled declaratively, but punctuation nodes would still appear). None of that is needed by semantic analysis or code generation - a `BINOP` node only needs its operator and two operand subtrees, not a matched pair of parenthesis tokens. Building an AST directly in the parser's semantic actions (each grammar rule constructs exactly one AST node from its already-reduced children) discards this syntactic noise at the point of creation, keeping the tree proportional to the program's actual structure rather than to its concrete grammar derivation.

4.3 Complexity Analysis: Memory and Construction Cost

Exactly one `ASTNode` is allocated per reduced grammar rule that produces a node (one per literal, identifier use, operator application, and statement). For a program whose token stream has length n, the number of AST nodes is **O(n)**, and since each node has a fixed number of fields (bounded by the largest union member), total AST memory is **O(n)**. Construction cost is **O(1)** per node (a single `malloc` and field assignment), so building the whole tree is **O(n)** overall, done incrementally as the parser reduces - there is no separate tree-construction pass.

5. Symbol Table and Semantic Analysis

5.1 Scope Management Approach

`symbol_table.c` implements scopes as a linked stack of hash tables. Each `Scope` owns a fixed-size array of 211 hash buckets (separate chaining, djb2 string hash) and a `parent` pointer to the enclosing scope. `enterScope()`/`exitScope()` push and pop a scope; `declareSymbol()` inserts into (and rejects a duplicate within) only the *current* scope's table; `lookupSymbol()` walks from the current scope outward to the global scope, returning the innermost match - which is exactly what makes an inner declaration correctly shadow an outer one of the same name.

5.2 Type-Checking Implementation

`semantic.c` performs a single recursive-descent walk over the AST (`checkStmt` / `checkExpr`), threading a `ValueType` (`TYPE_INT`, `TYPE_BOOL`, or `TYPE_UNKNOWN` for an already-erroneous subexpression, which suppresses cascading errors) bottom-up through every expression:

- Arithmetic operators (`+ - * /`) require both operands `int` and produce `int`.
- Order operators (`< >`) require both operands `int` and produce `bool`.
- Equality operators (`== !=`) require both operands to have the *same* type (either both `int` or both `bool`) and produce `bool`.
- Unary minus requires an `int` operand.

5.3 Error Detection Strategy

All four required semantic checks are implemented directly against the scoped symbol table:

- **Undeclared variables** - any identifier use where `lookupSymbol` returns `NULL`.
- **Multiple declarations in the same scope** - `declareSymbol` returns 0 when the name already exists in the *current* scope's bucket chain (an outer-scope match is a legal shadow, not an error).
- **Type mismatch in expressions** - checked at every binary operator, assignment, and initializer, as described above.
- **Invalid conditional expressions** - the condition of both `if` and `while` must type-check to `bool`.

Errors do not abort the walk: each is printed immediately with its source line and the walk continues, so a single compilation reports every semantic error found in one pass rather than only the first. Code generation only proceeds once the error count is zero.

5.4 Complexity Analysis: Lookup, Insertion, and Traversal Cost

With a fixed 211-bucket hash table per scope and the djb2 hash distributing reasonably uniformly, `declareSymbol` (insertion) is **O(1)** amortized within one scope. `lookupSymbol` is **O(1)** amortized per scope examined, and in the worst case examines every enclosing scope, giving **O(d)** where *d* is the current scope-nesting depth - in practice a small constant for MiniLang programs, since *d* is bounded by how deeply `{ }` blocks are nested. The semantic walk itself visits every AST node exactly once, performing O(1) symbol-table work per node, so full semantic analysis of a program with *n* AST nodes runs in **O(n*d)** time, which is effectively **O(n)** for realistic (shallow) nesting depths. Space usage is **O(v)** where *v* is the number of live variable declarations across all currently-open scopes (each popped scope's symbols

are freed immediately in `exitScope`).

6. Intermediate Code Generation

6.1 TAC Design and Instruction Format

`codegen.c` represents TAC as a dynamic array of a single `TACInstr` struct with a `kind` tag and `op / arg1 / arg2 / result` string fields, covering seven instruction kinds:

```
result = arg1                (TAC_ASSIGN, e.g.  x = t1)
result = arg1 op arg2         (TAC_BINOP,  e.g.  t1 = a + b)
result = op arg1              (TAC_UNOP,   e.g.  t2 = -t1)
result:                       (TAC_LABEL,  e.g.  $L1:)
goto result                   (TAC_GOTO)
ifz arg1 goto result          (TAC_IFZ)
print arg1                    (TAC_PRINT)
```

Fresh temporaries (`$t1, $t2, ...`) and labels (`$L1, $L2, ...`) are minted by simple monotonically-increasing counters. Every temporary produced while translating a subexpression is, by construction, consumed exactly once by its immediate parent - a property the optimizer's dead-code pass relies on.

6.2 Translation of Conditionals and Loops

```
if (cond) S1 else S2:            while (cond) S:
    <code to eval cond -> c>          L_start:
    ifz c goto L_false                    <code to eval cond -> c>
    <code for S1>                         ifz c goto L_end
    goto L_end                            <code for S>
    L_false:                              goto L_start
    <code for S2>                     L_end:
    L_end:
```

Both forms use exactly the textbook back-patched-label scheme, generalized so the `if`-without-`else` case simply omits the `goto L_end` / second block and reuses `L_false` as the join point.

6.3 Scope-Correct Naming (Shadowing)

Block scoping means two distinct declarations can share the same source name (an inner `int x` shadowing an outer one). TAC itself has no concept of scope, so `codegen.c` mirrors semantic.c's scope stack with its own lightweight declare/resolve mechanism: the first declaration of a name in the currently active scope chain keeps its original name as its TAC storage location; a shadowing declaration is assigned a disambiguated name (`x$2`, `x$3`, ...). Every identifier use resolves through this table to the innermost visible declaration's TAC name, exactly matching `lookupSymbol`'s resolution order, so the generated code observes the same shadowing behavior the semantic analyzer already validated.

6.4 TAC-Level Optimization

`optimizeTAC()` runs four passes to a fixed point (capped at 8 iterations, since folding can expose new simplification opportunities and vice versa):

- **Constant folding** - a `BINOP`/`UNOP` whose operand(s) are literal integers is evaluated at compile time and rewritten to a plain `ASSIGN` of the computed literal (division by a literal zero is left unfolded, since that is a runtime error condition, not a compile-time constant).

- **Algebraic simplification** - identity/absorbing-element patterns (`x+0, 0+x, x-0, x*1, 1*x, x*0, 0*x, x/1`) collapse a `BINOP` into a plain copy.
- **Local copy propagation** - a small alias table maps a temporary/variable to the value it was just copied from; later uses are substituted through the (path-compressed) chain, and the table is invalidated at every label (a conservative, correct choice, since a label is a control-flow merge point where the straight-line assumption no longer holds) and whenever the aliased name is redefined.
- **Redundant-temporary / dead-code elimination** - since every generated temporary is consumed exactly once, a single linear scan collects every name still referenced anywhere in the surviving code; any instruction defining a temporary that is not in that set is marked dead and dropped from further output. User-named variables are always kept (they are not subject to this pass), since a full whole-program liveness analysis across arbitrary control flow was judged out of scope for this project; this is a deliberate, documented simplifying assumption.

6.5 Complexity Analysis: Cost of AST Traversal and Code Emission

TAC generation walks the (already-built) AST exactly once, emitting a constant number of instructions per node (a `BINOP` emits one instruction; an `if`/`while` emits a small constant number of label/jump instructions in addition to its children's code). For *n* AST nodes this is **O(n)** time and produces **O(n)** TAC instructions. Each of the four optimization passes is a single linear scan over the instruction list - **O(m)** per pass for *m* instructions, with the fixed-point loop bounded by a constant (8) iteration cap, so optimization is **O(m)** overall (constant-factor, not growing with input). Target code generation is likewise one linear pass emitting a constant number of stack-machine instructions per TAC instruction, **O(m)**.

7. Overall Compiler Complexity

7.1 Combined Time Complexity of All Phases

Phase	Time Complexity	Dominant Cost
Lexical analysis	O(n)	n = source characters
Syntax analysis	O(n)	n = tokens (LALR table-driven)
AST construction	O(n)	one node per reduced construct
Semantic analysis	O(n·d)	d = max scope-nesting depth (effectively O(n))
TAC generation	O(n)	one AST walk
TAC optimization	O(m)	m = TAC instructions, O(n); constant-bounded passes
Target code generation	O(m)	one TAC walk

Since every phase is linear (or effectively linear, treating scope-nesting depth as a small constant bounded by realistic program structure) in the size of its input, and each phase's input size is itself O(n) in the original source length, the **compiler as a whole runs in O(n) time** for a source program of *n* characters.

7.2 Space Usage of AST, Symbol Table, and TAC

- **AST:** O(n) nodes, each of fixed size - O(n) total.
- **Symbol table:** O(v) live symbols at any point in the walk, where v is the number of variables declared in currently-open scopes; peak total across the whole compilation is O(V) where V is the total number of declarations in the program.
- **TAC:** O(n) instructions (bounded by the AST size that generated them), stored in a dynamic array that doubles on growth, giving O(1) amortized append and O(n) total space.

7.3 Discussion: Does the Compiler Operate in Linear Time?

Yes, for realistic MiniLang programs. Every phase is either strictly O(n) (lexing, parsing, AST construction, TAC generation, target codegen) or O(n*d) with d bounded by scope-nesting depth (semantic analysis) - and d grows with how deeply the programmer chooses to nest { } blocks, not with program length, so it is effectively a small constant in practice (single-digit nesting is typical). The TAC-optimization fixed-point loop is capped at a constant number of iterations rather than being allowed to scale with program size, which keeps it from becoming a hidden superlinear cost. There is no phase in this implementation that performs quadratic or worse work as a function of source size (e.g. no $O(n^2)$ symbol lookup, no repeated whole-program rescans), so the compiler is linear end-to-end.

8. Sample Compilation Results

8.1 Example MiniLang Program (testcases/valid1_features.ml)

```
// Exercises every supported MiniLang construct.
int a;
int b;
int sum;
bool flag;

a = 5;
b = 10;
sum = a + b * 2;
flag = sum > 20;

if (flag) {
    print(sum);
} else {
    print(0);
}

int i;
i = 0;
while (i < 3) {
    print(i);
    i = i + 1;
}
```

8.2 Generated Three-Address Code (unoptimized)

```
a = 5
b = 10
$t1 = b * 2
$t2 = a + $t1
sum = $t2
$t3 = sum > 20
flag = $t3
ifz flag goto $L1
print sum
goto $L2
$L1:
print 0
$L2:
i = 0
$L3:
$t4 = i < 3
ifz $t4 goto $L4
print i
$t5 = i + 1
i = $t5
goto $L3
$L4:
```

8.3 Optimized Three-Address Code (written to output.tac)

```
a = 5
b = 10
sum = 25
flag = 1
ifz 1 goto $L1
```

```
print 25
goto $L2
$L1:
print 0
$L2:
i = 0
$L3:
$t4 = i < 3
ifz $t4 goto $L4
print i
$t5 = i + 1
i = $t5
goto $L3
$L4:
```

Constant folding collapses `b * 2` (10*2) and `a + $t1` (5+20) directly to `sum = 25`, and `sum > 20` folds to `flag = 1`; the now-constant branch condition and the loop body (whose trip count depends on a runtime variable, `i`) are left as genuine control flow, as required.

8.4 Generated Target (Stack-Machine) Code (written to output.asm)

```
LOAD  5       STORE a       LOAD  10      STORE b
LOAD  25      STORE sum     LOAD  1       STORE flag
LOAD  1       JZ    $L1     LOAD  25      PRINT
JMP   $L2     $L1:          LOAD  0       PRINT
$L2:          LOAD  0       STORE i       $L3:
LOAD  i       LOAD  3       LT            STORE $t4
LOAD  $t4     JZ    $L4     LOAD  i       PRINT
LOAD  i       LOAD  1       ADD           STORE $t5
LOAD  $t5     STORE i       JMP   $L3     $L4:
```

(Reflowed onto four columns here to fit the page; the actual output.asm has one instruction per line.) Every TAC instruction maps mechanically to LOAD/STORE around one operation, e.g. t = a op b -> LOAD a; LOAD b; OP; STORE t - which sidesteps register allocation entirely by treating every temporary and variable as an addressable storage slot.

8.5 Verified Error Handling

Each error class was exercised against the actual built `minicompiler.exe` (MSYS2 gcc 16.1.0 / flex 2.6.4 / bison 3.8.2) and produced the expected diagnostic with a correct exit code of 1:

Testcase	Error Type	Sample Output
err_lexical.ml	Lexical	Lexical Error at line 2: unexpected character '@'
err_syntax.ml	Syntax	Syntax Error at line 2: syntax error, unexpected IDENT, expecting ASSIGN or SE
err_semantic_undeclared.ml	Semantic	Semantic Error at line 2: undeclared variable 'b'
err_semantic_redeclare.ml	Semantic	Semantic Error at line 2: variable 'a' is already declared in this scope
err_semantic_typemismatch.ml	Semantic	Semantic Error at line 3: type mismatch: cannot assign bool value to int varia
err_semantic_condition.ml	Semantic	Semantic Error at line 6: invalid conditional expression: expected bool, got i

8.6 Verified Scope Shadowing (testcases/valid2_scoping.ml)

This test specifically exercises the codegen-level scope handling described in §6.3:

```
int x;  x = 1;
{ int x;  x = 99;  print(x); }
print(x);

--- TAC ---            --- Optimized ---
x = 1                   x = 1
x$2 = 99                x$2 = 99
print x$2               print 99
print x                 print 1
```

The inner block's `x` is correctly disambiguated to `x$2`, so the two `print` statements resolve to their own distinct variables and correctly print `99` then `1`, matching MiniLang's block-scoping semantics.